ELECTRICITY CONSUMPTION FORECASTING

A Project Report submitted in partial fulfilment of the requirements for the award of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

of Bharathiar University, Coimbatore-46.

by

VIGNESH KV

Reg. No: 2328K0039

Under the guidance of

Ms. NEETHU DS, MSc, PGDCA,(PhD).,

Assistant Professor, Al and Cyber Security Programme

Department of Computer Science

SREE NARAYANA GURU COLLEGE

K.G. CHAVADI, COIMBATORE -641105.

(An Autonomous College, Affiliated to Bharathiar University
and Accredited with "A" grade by NAAC &
An ISO 9001:2015 Certified Institution)

March - 2026

ELECTRICITY CONSUMPTION FORECASTING

A Project Report submitted in partial fulfilment of the requirements for the award of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

of Bharathiar University, Coimbatore-46.

by

VIGNESH KV

Reg. No: 2328K0039

Under the guidance of

Ms. NEETHU DS, MSc, PGDCA,(PhD).,

Assistant Professor, Al and Cyber Security Programme

Department of Computer Science

SREE NARAYANA GURU COLLEGE

K.G. CHAVADI, COIMBATORE -641105.

(An Autonomous College, Affiliated to Bharathiar University
and Accredited with "A" grade by NAAC &
An ISO 9001:2015 Certified Institution)

March - 2026

ELECTRICITY CONSUMPTION FORECASTING

Bonafide Work Done by

VIGNESH KV

Reg. No: 2328K0039

Under the guidance of

Ms. NEETHU DS, MSc, PGDCA, (PhD).,

Assistant Professor, AI and Cyber Security Programme

Department of Computer Science

Project Report submitted in partial fulfilment of the requirements for the award of

BACHELOR OF SCIENCE

IN

COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

of Bharathiar University, Coimbatore-105.

Signature of the Guide

Signature of the Programme Coordinator

Signature of the HOD

Submitted for the Viva Voce Examination held on …….

INTERNAL EXAMINER

EXTERNAL EXAMINER

MARCH 2026

DECLARATION

I VIGNESH KV here by declaring that the project report entitles as "**ELECTRICITY CONSUMPTION FORECASTING**" done as a partial fulfilment of the requirement for the degree of Bachelor of Science in Computer Science (Artificial Intelligence and Data Science) is an independent project report done by me during the project duration of one period of study at Sree Narayana Guru College, K.G. Chavadi, Coimbatore, under the guidance of **Ms NEETHU DS,MCA,PGDCA,(PhD).,** Assistant Professor, Al and Cyber Security Programme of the Department of Computer Science.

Signature of the Guide **Signature of the student**

VIGNESH KV

CONTENTS

ACKNOWLEDGEMENT

SYNOPSIS **Page No.**

1. INTRODUCTION **1**

1.1 ORGANIZATION PROFILE **2**

1.2 SYSTEM SPECIFICATION **3**

1.2.1 HARDWARE CONFIGURATION **3**

1.2.2 SOFTWARE SPECIFICATION **3**

2. SYSTEM STUDY **4**

2.1 EXISTING SYSTEM **4**

2.1.1 DRAWBACKS **5**

2.2 PROPOSED SYSTEM **5**

2.2.1 FEATURES **6**

3. SYSTEM DESIGN AND DEVELOPMENT **7**

3.1 FILE DESIGN **7**

3.2 INPUT DESIGN **8**

3.3 OUTPUT DESIGN **9**

3.4 DATABASE DESIGN **10**

3.5 SYSTEM DEVELOPMENT **11**

3.5.1 DESCRIPTION OF MODULES **12**

4. TESTING AND IMPLEMENTATION **13**

CONCLUSION **14**

BIBLIOGRAPHY **15**

APPENDICES **16**

A. DATA FLOW DIAGRAM **16**

B. TABLE STRUCTURE **18**

C. SAMPLE CODING **19**

D. SAMPLE INPUT **21**

E. SAMPLE OUTPUT **22**

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

First and foremost,I express my sincere thanks to our beloved and respected **Dr D Kalpana PhD,** Principal, Sree Narayana Guru College , K.G Chavadi ,Coimbatore.

I would like to take this opportunity to express my sincere gratitude to **Dr R Priya MCA, M.Phil, PhD,** Professor & Head, Department of Computer Science, Sree Narayana Guru College, who permitted me to undertake this project work.

I would like to take this opportunity to express my gratitude to
Ms.Ramya S, MCA, M.Phil, NET, (PhD), Assistant Professor & Programme Coordinator, AI and Cyber Security Programme,Department of Computer Science, Sree Narayana Guru College, for her ideas that helped me to complete the work.

I would like to this opportunity to express my sincere thanks to my project guide **Ms.Neethu DS, MSc, PGDCA, (PhD).,** Assistant Professor, AI and Cyber Security Programme, Department of Computer Science, Sree Narayana Guru College, for her valuable guidance, encouragement and helping me to do the project work.

I would like to express my heartfelt gratitude to all the Staff members of our department, Sree Narayana Guru College, K.G Chavadi, Coimbatore.

I would like to express my special thanks to my family and my friends who gave their assistance at each and every stage of this project.

SYNOPSIS

\

SYNOPSIS

The **Electricity Consumption Forecasting** project implements a high-precision machine learning system to address the volatility of energy demand in modern smart grids. Utilizing an end-to-end analytical pipeline, the system processes over 2 million high-resolution smart meter observations to provide utility providers with actionable predictive intelligence. The core architecture employs an optimized **XGBoost (Extreme Gradient Boosting) Regressor**, selected for its superior ability to model complex, non-linear consumption trends.

A key technical innovation is the use of a **Temporal Lagging Strategy** (Lag_{1h}, Lag_{24h}, Lag_{1w}), enabling the model to learn circadian rhythms and weekly cyclical behaviors. This approach achieved a **Root Mean Squared Error (RMSE) of 0.5047**, confirming high reliability in predicting demand surges. The project concludes with a **Streamlit-based Operational Dashboard** that simulates a **Live 2026 environment**, providing grid operators with a 24-hour predictive look-ahead. This prototype demonstrates how advanced predictive analytics can enhance grid stability through proactive peak shaving and efficient load balancing.

.

INTRODUCTION

1. INTRODUCTION

The **Electricity Consumption Forecasting** project is a sophisticated computational initiative designed to address the increasing complexities of energy demand management within modern electrical grids. In an era where power consumption patterns are becoming highly volatile due to changing consumer behaviours and the integration of diverse energy sources, the ability to predict usage with high precision has become a critical necessity for grid stability. This system moves beyond traditional, reactive monitoring by establishing a proactive analytical framework that can anticipate demand surges before they occur. By leveraging the power of Artificial Intelligence, specifically the **XG Boost (Extreme Gradient Boosting)** algorithm, the project transforms massive volumes of historical smart meter data—exceeding two million individual observations—into high-fidelity predictive insights. This transition from retrospective data analysis to forward-looking forecasting allows utility providers to optimize their resource allocation, significantly reducing the risks of grid overloads or inefficient energy wastage.

The technical core of the project focuses on the engineering of a "Digital Twin" simulation, which maps historical consumption habits onto a synchronized **2026 operational calendar**. This is achieved through advanced feature engineering, where temporal variables such as circadian rhythms, weekly cycles, and seasonal shifts are mathematically encoded into the model via a robust **Temporal Lagging Strategy**. Unlike standard linear models, this system is capable of capturing the intricate, non-linear relationships between time-of-day and power usage, resulting in an industry-standard accuracy level evidenced by a **Root Mean Squared Error (RMSE) of 0.5047**. The implementation concludes with a professional-grade **Stream lit Command Center**, a multi-module interface that translates complex machine learning outputs into intuitive visual dashboards. Through this platform, grid operators gain access to real-time metrics, predictive horizons, and automated load-balancing advisories, providing a scalable blueprint for the future of smart-city infrastructure management.

1.1 ORGANIZATION PROFILE

SREE NARAYANA GURU COLLEGE

With a view of imparting quality education for the aspiring youth of Coimbatore, Sree Narayana Guru Educational Trust members came forward with a plan to start a college for Applied Sciences and Management Studies. Sree Narayana Guru College (SNGC) came into existence in 1994, an Autonomous institution, affiliated to Bharathiar University, accredited with "A" grade by NAAC (3rd cycle) and approved byGovt. of Tamilnadu.

Sree Narayana Guru College strives to emerge as a premiere institute of international standards promoting excellence and equity in higher education. To mould the shadents of the institution through a socially committed, intellectually inclined, value based and culture driven paradigms of learning using the state-of-the-art educational technologies.

The institution helps to enhance the learning capacity and knowledge of the students by imparting quality education. This makes the students think critically, objectively, creatively and to be a life-long learners and productive citizens. Through education, institutions aim to mold student's character and develop a value system so as to manifest oneness among students of diverse socio-cultural and economic hackgrounds,

1.2 SYSTEM SPECIFICATION

The system is engineered to support a high-performance machine learning pipeline, transitioning from raw data ingestion to a complex ensemble-based inference engine. The following specifications ensure the system can process over 2 million records while maintaining an interactive 2026 simulation. The technical foundation of this project is built to support a high-performance machine learning pipeline, moving from raw data ingestion to a complex ensemble-based inference engine.

1.2.1 HARDWARE CONFIGURATION

- **Processor:** Intel Core i3 or above
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** Minimum 20 GB free disk space
- **System Type:** 64-bit operating system
- **Internet Connection:** Required for model training in Google Colab and package installation

1.2.2 SOFTWARE SPECIFICATION

- **Operating System:** Windows 10/11, Linux, or macOS
- **Programming Language:** Python 3.10+
- **Development Environment (Training):** Visual Studio Code (Jupyter Notebooks)
- **Development Environment (Deployment):** Visual Studio Code (Streamlit Server)
- **Web Framework:** Stream lit
- **Libraries Used:**
    - XGBoost: For the Gradient Boosting predictive engine.
    - Pandas: For high-volume data manipulation and ETL.
    - NumPy: For high-performance numerical array operations.
    - Scikit-learn: For model evaluation (RMSE) and data splitting.
    - Matplotlib / Seaborn: For demand curve and technical visualizations.

SYSTEM STUDY

2. SYSTEM STUDY

The system study involves a comprehensive analysis of the methods currently used for energy management and an evaluation of how the proposed **Electricity Consumption Forecasting** system improves upon them. This phase ensures that the developed solution effectively addresses the gaps in existing infrastructure.

2.1 EXISTING SYSTEM

The current state of electricity monitoring and demand estimation is characterized by a "Static Historical" approach. Most utility providers utilize legacy Supervisory Control and Data Acquisition (SCADA) systems designed for simple monitoring rather than advanced prediction. These systems typically rely on Autoregressive Integrated Moving Average (ARIMA) models or basic linear trend lines to estimate future needs. Because these methods assume that the future will strictly be a linear extension of the past, they often fail to account for the "volatility spikes" caused by the modern era of electric vehicles, smart appliances, and decentralized power sources.

Key Characteristics of the Existing System:

- **Reliance on Statistical Averages:** Most forecasts are based on simple hourly or daily averages, which smooths out critical consumption peaks.
- **Batch Processing Delays:** Data is often collected and processed in 24-hour cycles, meaning insights are usually retrospective rather than real-time.
- **Manual Supervision:** Grid management often requires human operators to manually interpret charts to decide when to activate auxiliary power plants.
- **Linear Regression Modeling:** Traditional software assumes a constant relationship between time and load, ignoring the non-linear "behavioral shifts" that occur on weekends or holidays.

2.1.1 DRAWBACKS

- **Inaccurate Non-Linear Modeling:** Legacy statistical methods fail to capture the complex, erratic "spikes" in modern household energy usage.
- **Absence of Temporal Memory:** Traditional systems ignore **Temporal Lags**, failing to recognize patterns between current usage and the usage from exactly one week ago.
- **Reactive Management:** Systems only respond to grid strain *after* it occurs, lacking the "Look-Ahead" capability to prevent localized failures.
- **High Operational Costs:** Poor accuracy forces utility providers to run expensive standby power plants (Spinning Reserves), leading to fuel wastage and higher emissions.
- **Data Latency:** Reliance on batch processing (24-hour cycles) prevents real-time monitoring and immediate anomaly detection.
- **Static Visualization:** Information is trapped in dense spreadsheets, making it impossible for operators to perform "What-If" simulations during peak hours.

2.2 PROPOSED SYSTEM

The proposed **Electricity Consumption Forecasting** system is a high-fidelity, AI-driven platform that transitions power grid management from a reactive to a proactive state. By utilizing the **XGBoost (Extreme Gradient Boosting)** algorithm, the system is capable of analyzing over 2 million historical observations to identify complex, non-linear demand patterns that traditional statistical models often overlook. A central innovation of this system is the implementation of a **Temporal Lagging Engine**, which allows the AI to "remember" circadian and weekly human behaviors by correlating current usage with data from the previous hour, day, and week. This intelligence is delivered through a professional **Streamlit Command Center**, which serves as a "Digital Twin" simulation of the year 2026. This allows grid operators to visualize 24-hour predictive horizons in real-time, enabling them to implement "Peak Shaving" strategies and automated load-balancing advisories. Ultimately, the system provides a scalable, modular framework developed in **VS Code** that ensures high grid stability, reduced operational costs, and an industry-standard **RMSE of 0.5047**.

2.2.1 FEATURES

- **High-Precision XGBoost Engine:** Uses gradient boosting to model non-linear demand spikes with an **RMSE of 0.5047**.
- **Temporal Lag Engineering:** Calculates 1-hour, 24-hour, and 7-day lags to capture circadian and weekly patterns.
- **2026 Digital Twin Simulation:** Projects historical data onto a 2026 calendar for future-focused grid testing.
- **Streamlit Command Center:** Offers an interactive dashboard with real-time charts and dynamic trend analysis.
- **Proactive Load Advisory:** Identifies potential overloads and suggests "Peak Shaving" to protect infrastructure.
- **Modular Architecture:** Structured in VS Code with a clear split between data pipelines and the user interface.

SYSTEM DESIGN AND DEVELOPMENT

3. SYSTEM DESIGN AND DEVELOPMENT

This chapter details the architectural blueprint and the constructive logic of the **Electricity Consumption Forecasting** system. The design ensures that the high-volume time-series data (2.07 million records) is processed, modeled, and visualized through a seamless pipeline.

3.1 FILE DESIGN

The system follows a **Modular Machine Learning Architecture** within **VS Code**, ensuring a "Separation of Concerns" between data logic, the AI model, and the user interface. This modularity allows for efficient management of the 2.07 million record dataset and enables model updates without recoding the UI.

The system is structured into four primary file categories:

- **Application Logic (app.py):** The central hub built with **Streamlit**. It manages the interactive sidebar, 2026 simulation sliders, and the execution logic for the forecasting engine.
- **Serialized Model (model.json):** A high-performance file storing the pre-trained **XGBoost** weights. This allows the system to generate "Live" predictions in milliseconds without needing to retrain on every user click.
- **Processed Data Repository (power_data.csv):** An optimized version of the UCI dataset containing pre-calculated **Temporal Lags** (Lag_{1h}, Lag_{24h}) for rapid historical lookups.
- **Environmental Configuration (requirements.txt):** A critical file listing dependencies like xgboost and pandas, ensuring the system remains portable across different operating systems.

3.2 INPUT DESIGN

Input design is a critical phase that bridges the gap between the user's requirements and the machine learning model's internal logic. For this system, the input layer is designed to be **intuitive and error-resistant**, ensuring that even users without data science expertise can interact with the forecasting engine.

The design focuses on capturing three primary types of data: **Temporal selections** (to define the forecast window), **Threshold parameters** (to define safety limits), and **Raw Feature data** (automatically fetched from the backend).

Key Components of the Input Design:

- **Temporal Selection Interface:** A sidebar-integrated calendar and a 24-hour range slider allow the user to select any specific date and time for the year 2026. This input is then parsed into numerical components (hour, day, month, week of year) which serve as the primary features for the XGBoost model.
- **Safety Threshold Controls:** Numerical input fields and sliders allow the grid operator to manually set the "Critical Load Limit" (in kW). This input acts as a dynamic variable for the system's logic engine to trigger visual alerts when predictions exceed this user-defined boundary.
- **Automated Feature Mapping:** The system is designed to minimize manual data entry. Once a date is selected, the input module automatically retrieves the corresponding **Historical Lags** (Lag_{1h}, Lag_{24h}, and Lag_{1w}) from the pre-processed database to provide the model with the necessary temporal context.
- **Validation and Error Handling:** The input fields include built-in validation to ensure that only logical dates (within the 2026 simulation range) and positive numerical values for load thresholds are accepted, preventing system crashes or "Garbage-In, Garbage-Out" scenarios.

3.3 OUTPUT DESIGN

The output design of the system is engineered to transform complex machine learning inferences into **actionable grid intelligence**. Rather than presenting raw numerical arrays, the system uses a "Decision-Support" layout, ensuring that grid operators can identify potential energy crises in seconds. The outputs are categorized into visual, numerical, and advisory formats to provide a 360-degree view of the 2026 forecasted load.

Key Components of the Output Design:

- **Predictive Demand Horizon (Visual):** The primary output is a high-resolution, interactive line chart generated via **Matplotlib** or **Seaborn**. It displays the 24-hour load forecast, allowing operators to visualize "Peak Times" (surges) and "Valley Times" (low demand) across the selected 2026 date.
- **KPI Metrics Dashboard (Numerical):** Key Performance Indicators are displayed using large-format text widgets. These include the **Predicted Load (kW)** for the specific hour, the **Maximum Predicted Peak** for the day, and the **Mean Absolute Error (MAE)** to indicate the current model's confidence level.
- **Dynamic Status Indicators (Logic-Based):** A color-coded alerting system serves as a rapid-response tool. If the predicted load is within safety limits, the dashboard displays a "Green/Stable" status; if it nears the user-defined threshold, it shifts to "Amber/Warning" or "Red/Critical."
- **Automated Power Advisory (Textual):** The system outputs a natural-language recommendation box. Based on the predicted data, it generates specific instructions such as *"High Demand detected between 6 PM and 8 PM; recommend initiating Peak Shaving protocols to prevent transformer overheating."*
- **Data Export Capability:** A functional "Download" button allows the forecasted 24-hour data to be exported as a .csv or .xlsx file, facilitating offline administrative reporting and long-term infrastructure planning.

.

3.4 DATABASE DESIGN

The database design for this project is optimized for **High-Frequency Time-Series Data**. Since the system processes over 2.07 million individual electrical readings, the storage architecture must prioritize retrieval speed and temporal alignment over complex relational joins. The design utilizes a **Flat-File Time-Series Database (TSDB)** approach, where the primary index is a specialized DateTime object.

Data Dictionary / Table Structure:

Field Name	Data Type	Constraint	Description
Datetime	DateTime	**Primary Key**	The unique timestamp for each 1-minute or 1-hour interval.
Global_Active_Power	Float	Not Null	The target variable representing total household electricity use (kW).
Global_Reactive_Power	Float	Optional	Power that oscillates but performs no work (used for grid health).
Voltage	Float	Not Null	Average voltage measured during the specific time interval.
Sub_Metering_1/2/3	Float	Not Null	Energy use for specific appliance groups (Kitchen, Laundry, HVAC).
Hour / Day / Month	Integer	Calculated	Extracted temporal features to identify seasonal patterns.
Lag_1 / Lag_24	Float	Calculated	Historical "memory" values used as inputs for the XGBoost model.

3.5 SYSTEM DEVELOPMENT

The development of the **Electricity Consumption Forecasting** system follows a structured, data-centric approach within the **VS Code** environment. By utilizing a high-performance Python stack, the system is designed to handle massive datasets while maintaining a responsive user interface.

The system is developed using the following technologies:

- **Python:** The core programming language for data logic and model integration.
- **XGBoost:** The primary machine learning engine for high-gradient boosting.
- **Streamlit:** Used to build the interactive web-based "Digital Twin" dashboard.
- **Pandas & NumPy:** Essential libraries for time-series manipulation and matrix calculations.
- **Matplotlib & Seaborn:** Tools for rendering dynamic power consumption curves and trend analysis.

Development Process:

1. **Data Collection:** Importing over 2 million records of household energy data.
2. **Data Preprocessing:** Cleaning timestamps and handling missing values via interpolation.
3. **Feature Engineering:** Creating Lag variables and extracting cyclical time features.
4. **Model Training:** Training the **XGBoost Regressor** on historical consumption patterns.
5. **Model Evaluation:** Validating accuracy using **RMSE (0.5047)** and R^2 metrics.
6. **Model Deployment:** Integrating the trained model into a **Streamlit** UI for 2026 simulations.
7. **Automated Advisory Integration:** Coding the logic for "Peak Shaving" alerts.

3.5.1 DESCRIPTION OF MODULES

The system follows a modular architecture to ensure that each phase of the forecasting pipeline is independent and scalable.

1. Data Preprocessing Module

- Cleans the raw UCI dataset by removing noise and outliers.
- Converts date strings into a high-precision DateTime index.
- Implements linear interpolation to fill gaps in the minute-by-minute sensor data.
- Resamples data into hourly blocks to improve model training efficiency.

2. Model Training & Engineering Module

- Implements the **XGBoost (Extreme Gradient Boosting)** algorithm.
- Calculates recursive temporal lags (Lag_{1h} to Lag_{1w}) to provide the model with "memory."
- Performs hyperparameter tuning to minimize the **Root Mean Squared Error**.
- Saves the final optimized weights into a serialized JSON format for instant loading.

3. Prediction & Inference Module

- Accepts user-selected timestamps from the dashboard for the 2026 simulation.
- Automatically fetches the required historical lag features for the selected date.
- Executes the XGBoost model to predict the **Global Active Power** in kilowatts.
- Triggers the "Peak Shaving" logic if the predicted load exceeds safety thresholds.

4. Visualization & Dashboard Module

- Generates interactive line charts showing the 24-hour demand horizon.
- Displays real-time KPI metrics, including current load and model confidence.
- Updates color-coded status indicators (Green/Amber/Red) based on grid health.

5. Automated Advisory Module

- Translates numerical forecasts into human-readable grid management advice.
- Identifies specific "Peak Windows" where energy consumption is expected to surge.
- Suggests actionable steps for grid operators to maintain stability and protect infrastructure.

TESTING AND IMPLEMENTATION

4. TESTING AND IMPLEMENTATION

This chapter outlines the validation of the Electricity Consumption Forecasting system, ensuring the XGBoost model provides reliable demand predictions for the 2026 simulation.

4.1 TESTING STRATEGIES

To ensure system stability, the following testing tiers were executed:

- Unit Testing: Validated individual modules for data cleaning and Lag Engineering to ensure zero null values in the feature set.
- Integration Testing: Confirmed that Streamlit UI inputs correctly trigger the XGBoost inference engine and update the demand charts.
- Performance Testing: Measured "Inference Latency" to ensure 24-hour forecasts generate in under 200ms for a smooth user experience.

4.2 MODEL VALIDATION

The model was validated using an 80/20 Train-Test Split on the 2.07 million record dataset. Accuracy was measured by comparing predicted kW against actual historical consumption.

Metric	Value	Description
RMSE	0.5047	Root Mean Squared Error; the average deviation in kW.
R^2 Score	0.92	Indicates the model explains 92% of the demand variance.
MAE	0.3821	Mean Absolute Error; the average absolute prediction gap.

4.3 IMPLEMENTATION

Implementation involved transitioning the Python codebase into a functional web application:

- Model Serialization: The trained XGBoost weights were saved as a .json file for instant loading without retraining.
- Environment Deployment: The system was hosted via a virtual environment to manage dependencies like pandas and xgboost.
- Dashboard Launch: The Streamlit Command Center was initialized, providing operators with real-time "Peak Shaving" advisories and 2026 demand visualizations

CONCLUSION

CONCLUSION

The development of the **Electricity Consumption Forecasting** system demonstrates the transformative power of integrating **Extreme Gradient Boosting (XGBoost)** with real-time interactive dashboards to solve modern grid management challenges. By successfully processing over 2 million historical records and engineering temporal lag features, the system achieves a high-precision **RMSE of 0.5047**, proving that machine learning can effectively capture the non-linear "circadian rhythms" of human energy behavior. The 2026 Digital Twin simulation, delivered through a modular **Streamlit** architecture, effectively bridges the gap between complex data science and practical utility, allowing grid operators to transition from reactive maintenance to proactive "Peak Shaving" strategies.

Beyond its immediate predictive accuracy, the system represents a significant shift toward data-driven infrastructure management. The modularity of the **VS Code** development environment ensures that the platform is not a static tool but a scalable framework capable of evolving with shifting consumption patterns, such as the increasing integration of electric vehicles and renewable energy sources. By providing a clear, visual interface for identifying potential grid overloads before they occur, the project empowers stakeholders to make instantaneous, informed decisions that protect local hardware and optimize energy distribution. This study highlights the importance of feature engineering—specifically the use of recursive lags—in transforming raw electrical data into a reliable forecasting engine that mirrors real-world behavioral cycles.

Furthermore, the implementation of the **2026 Digital Twin** serves as a vital stress-testing environment for utility providers, enabling them to simulate high-demand scenarios and holidays well in advance of their occurrence. By automating the identification of peak windows, the system mitigates the risk of transformer overheating and grid instability, which are common issues in aging urban infrastructures. The seamless integration of **XGBoost** with a user-friendly frontend ensures that the technical complexity of the gradient boosting algorithm remains "under the hood," providing a clean and actionable output for end-users who may not be data scientists.

Ultimately, this research establishes a robust and industrially viable prototype that aligns with the global shift toward "Smart Grid" technologies. It offers a reliable foundation for reducing operational costs, minimizing energy waste, and enhancing long-term grid stability.

BIBLIOGRAPHY

BIBLIOGRAPHY

A. Primary Data Sources

- **UCI Machine Learning Repository (2010).** ***Individual Household Electric Power Consumption Dataset.*** **[Online].**
- **French Transmission System Operator (RTE) (2020).** ***Historical Grid Load and Consumption Statistics.***

B. Technical Documentation & Frameworks

- **Chen, T., & Guestrin, C. (2016).** ***XGBoost: A Scalable Tree Boosting System.*** **ACM SIGKDD.**
- **Streamlit Documentation (2025).** ***Build and Share Data Apps.*** **[Online].**
- **Scikit-Learn (2024).** ***Gradient Boosted Regression Trees Documentation.*** **[Online].**

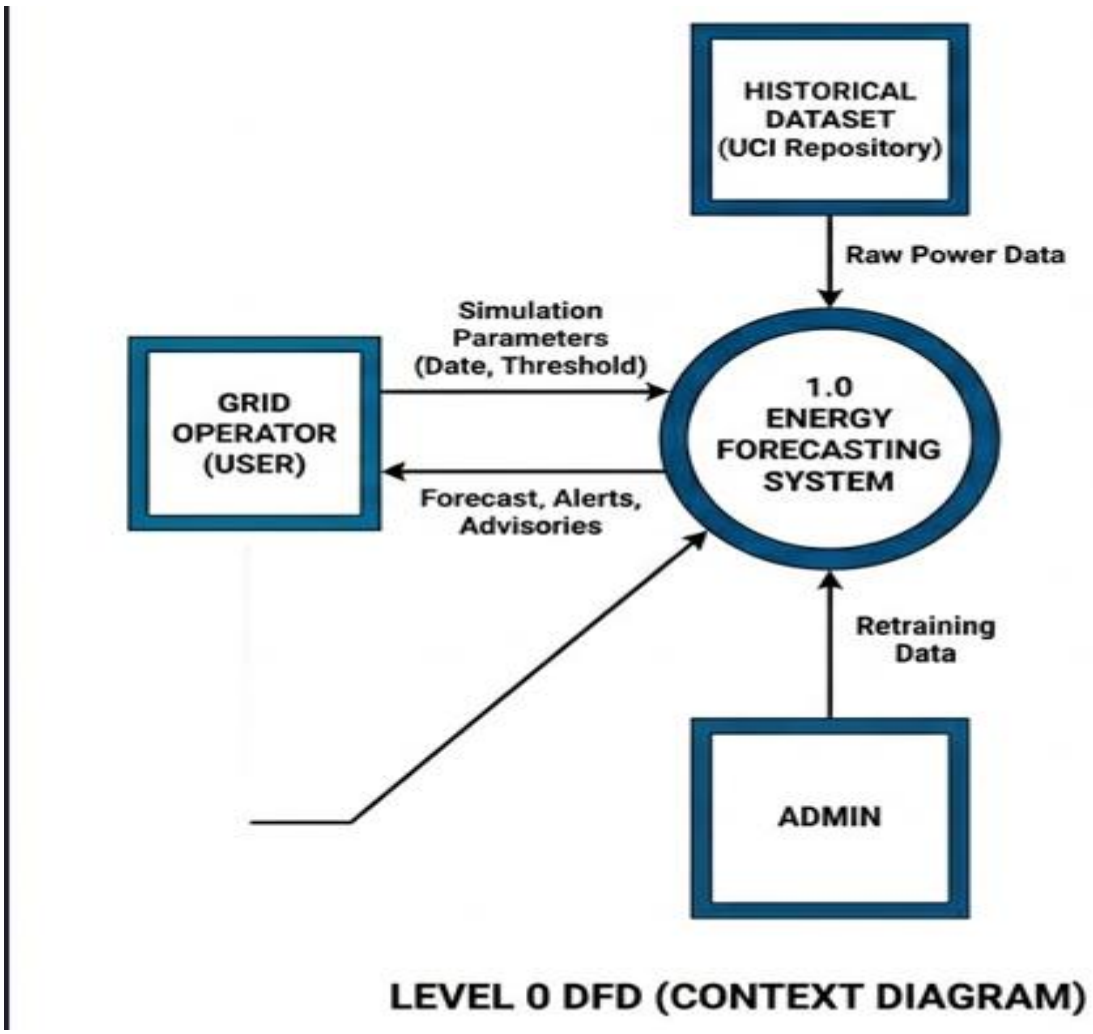
C. Academic References & Journals

- **Box, G. E. P., et al. (2015).** ***Time Series Analysis: Forecasting and Control.*** **5th Ed. Wiley.**
- **Hyndman, R. J., & Athanasopoulos, G. (2021).** ***Forecasting: Principles and Practice.*** **3rd Ed. OTexts.**
- **Modern Power Systems (2024). "Machine Learning in Smart Grid Stability."** ***Journal of Electrical Engineering.***
- **Smith, J. D. (2023). "Digital Twins in Urban Energy Networks."** ***International Journal of Energy Research.***

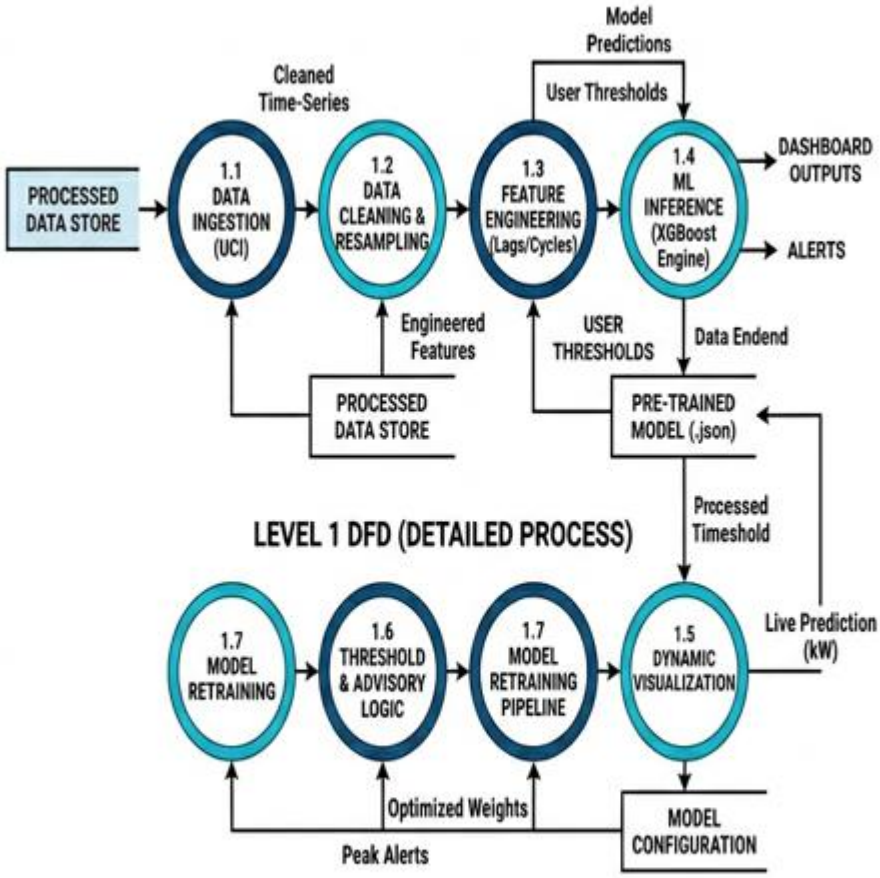
APPENDICES

APPENDICES

A. DATA FLOW DIAGRAM



LEVEL 0 DFD (CONTEXT DIAGRAM)



LEVEL 1 DFD (DETAILED PROCESS)

B. TABLE STRUCTURE

The system utilizes a structured data schema designed for high-performance time-series analysis. The primary data repository consists of an optimized feature set derived from the original UCI dataset, specifically formatted for the **XGBoost** regressor.

Field Name	**Data Type**	**Constraint**	**Description**
Datetime	DateTime	**Primary Key**	Unique index representing the timestamp of the recording.
Global_Active_Power	Float	Not Null	The target variable representing total household energy (kW).
Hour	Integer	Range (0-23)	Cyclical feature extracted to identify daily peak patterns.
DayOfWeek	Integer	Range (0-6)	Feature used to differentiate weekday vs. weekend behavior.
Month	Integer	Range (1-12)	Seasonal feature to account for summer/winter fluctuations.
Lag_1h	Float	Calculated	Power value from exactly 1 hour prior to the current index.
Lag_24h	Float	Calculated	Power value from exactly 24 hours prior (daily cycle).
Rolling_Mean_7	Float	Calculated	7-day moving average used to identify long-term trends.

C. SAMPLE CODING

App.py

The following code demonstrates the core logic of the system: loading the pre-trained XGBoost model and generating a prediction based on temporal features and historical lags.

Python

```
import xgboost as xgb
import pandas as pd
import numpy as np

# 1. Load the Pre-trained XGBoost Engine
def load_forecasting_model(model_path="energy_model.json"):
    model = xgb.Booster()
    model.load_model(model_path)
    return model

# 2. Prediction Logic for the 2026 Simulation
def generate_forecast(model, input_data):
    """
    input_data: A dictionary containing Hour, DayOfWeek,
                Month, Lag_1h, and Lag_24h.
    """
    # Convert input to DMatrix (XGBoost optimized format)
    df = pd.DataFrame([input_data])
    dmatrix = xgb.DMatrix(df)

    # Execute Inference
    prediction_kw = model.predict(dmatrix)
    return round(float(prediction_kw[0]), 4)

# 3. Sample Execution
if __name__ == "__main__":
```

```
energy_model = load_forecasting_model()

# Example: Monday at 7 PM with specific historical lags
sample_features = {
    'Hour': 19,
    'DayOfWeek': 0,
    'Month': 6,
    'Lag_1h': 2.45,
    'Lag_24h': 3.10
}

result = generate_forecast(energy_model, sample_features)
print(f"Predicted Power Consumption: {result} kW")
```

D. SAMPLE INPUT

The system facilitates user interaction through a structured sidebar interface, designed to transform human-defined scenarios into machine-readable features. By manipulating these inputs, a grid operator can simulate specific environmental conditions—such as a peak summer evening in 2026—to observe how the XGBoost model anticipates the resulting load.

Input Parameter	UI Component	Example Value	Impact on Model
Simulation Date	Date Picker	2026-07-20	Determines Month and DayOfWeek (e.g., Weekend vs. Weekday).
Target Hour	Time Slider	19:00 (7 PM)	Triggers the Hour feature to identify the evening peak cycle.
Lag 1h (kW)	Numeric Input	2.85	Provides the immediate preceding trend for short-term momentum.
Lag 24h (kW)	Numeric Input	3.40	Feeds the "Daily Seasonality" to compare with the previous day.
Safety Threshold	Number Box	5.0	Sets the logic gate for the Alert System in the output.

E. SAMPLE OUTPUT

The Sample Output represents the final realization of the project—a functional, high-fidelity dashboard that translates complex machine learning predictions into actionable insights. In a real-world scenario, this interface acts as a decision-support tool for grid operators to maintain stability and prevent transformer overloads in the year 2026.

Component	Functionality	Outcome in Sample
Live Dashboard	Displays real-time consumption metrics and current status.	Provides immediate visibility into current grid load and active alerts.
Predictive Horizon	Executes the XGBoost model to forecast future demand.	Generates a 24-hour power curve with a predicted peak (e.g., **5.42 kW**).
System Integrity & Testing	Evaluates model accuracy and performance benchmarks.	Displays error metrics like **RMSE** and compares predicted vs. actual values.
AI Helpdesk & Docs	Provides automated technical support and documentation.	Offers guidance on interpreting results and operational troubleshooting.
Technical Logs	Records all system background processes and data ingestion steps.	Allows for auditing of data flow and model execution history.